BOARD GAME CAFÉ

Document central de proiect: produs, arhitectură, infrastructură, model de date, securitate, AI, testare și decizii

VERSIUNE DOCUMENT	REFERINȚE DE BAZĂ	STADIU
Master v1.0 • 11.10.2026	Funcțional v1.1 / DB v0.13 / Arhitectură v0.1	Planificare; fără implementare

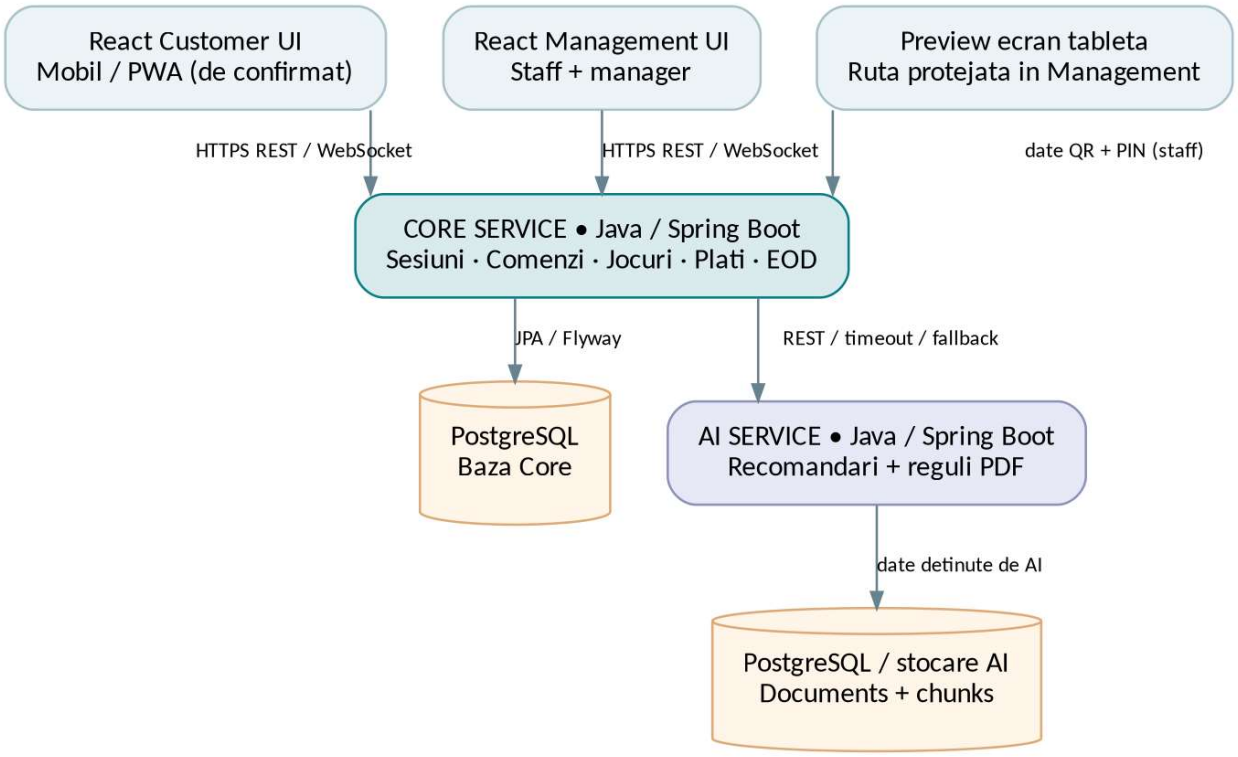


Figura 1. Arhitectura logică propusă: aplicații React, Core Service și serviciu AI.

REGULĂ DE GUVERNANȚĂ — „Confirmat” reprezintă o decizie luată. „Propus” este o recomandare tehnică neînghețată. „Deschis” necesită validare. „Future Work” este exclus din versiunea inițială. Documentul nu presupune existența unui produs deja implementat.

Referință academică: https://github.com/cuibus/SCDproiect1 (README, consultat 11 octombrie 2026).

Cuprins și controlul documentului

1. Context și obiective
2. Cerințe academice și trasabilitate
3. Domeniu, actori și permisiuni
4. Fluxuri funcționale și reguli
5. Stări și invarianti
6. Arhitectură și comunicare
7. Infrastructură și deploy
8. Model de date: inventar și relații
9. Dicționar complet de date
10. JPA, Code First și migrații
11. Securitate și identitate
12. Tranzacții și consistență
13. API și WebSocket planificate
14. Frontend și client mobil
15. Serviciul AI și documentele PDF
16. End of Day și analitice
17. Clean Code, SOLID și testare
18. Inițializare și demonstrație
19. Registrul deciziilor (ADR)
20. Riscuri, puncte deschise și Future Work
21. Plan de lucru și criterii de acceptare
Anexe: glosar și reguli detaliate

Controlul versiunilor

Versiune	Data	Descriere
Master v1.0	11.10.2026	Consolidarea tuturor deciziilor și propunerilor discutate până la modelul DB v0.13.
Următoarea	De stabilit	Rezolvarea punctelor deschise și promovarea schemei DB la v1.0.

Convenția de statut

Marcaj	Semnificație
CONFIRMAT	Decizie explicit acceptată în discuțiile proiectului.
DERIVAT	Consecință necesară pentru a respecta o decizie confirmată.
PROPUS	Soluție tehnică recomandată, dar încă neînghețată.
DESCHIS	Necesită o alegere sau confirmare.
FUTURE WORK	Nu intră în implementarea inițială.
RESPINS / ÎNLOCUIT	Propunere anterioară anulată de o decizie mai recentă.

1. Context și obiective

Board Game Café este o aplicație distribuită pentru administrarea unei cafenele cu jocuri de societate. Clienții intră la masă fără cont folosind QR și PIN, comandă produse, solicită jocuri și achită împreună nota. Staff-ul procesează comenzile și împrumuturile; managerul configurează sistemul și simulează zile de activitate pentru rapoarte comparative.

Scop demonstrativ: proiect universitar SCD, realizat individual, cu accent pe funcționalitate completă, înțelegerea fiecărei clase și demonstrarea cerințelor tehnice.

Constrângeri de lucru: Windows, cunoștințe Java și JavaScript, iPhone pentru clientul mobil, buget lunar redus, orizont estimat de 1–2 luni.

Principiu: design și justificări înainte de implementare; când scriem cod, acesta trebuie explicat și testat, cu Clean Code și SOLID fără supraproiectare.

1.1. Scope: ce intră în v1

- Sesiuni temporare per masă, acces QR/PIN și preview de tabletă în browserul staff-ului.
- Meniu, coș individual, comenzi individuale cu acceptare/respingere pe poziții și notă comună.
- Catalog de jocuri cu categorii multiple, exemplare fizice, rezervare, livrare, anulare și returnare.
- Tarif fix per persoană fizică participantă; checkout, redeschidere controlată și plată integrală simulată.
- Manager: utilizatori, mese, catalog, meniuri, setări, program informativ, rapoarte zilnice și export PDF.
- AI: recomandări explicate și asistent pentru reguli din PDF; fallback la filtrare deterministă pentru recomandări.
- Notificări în timp real și recuperare prin REST după reconectare; zile simulate succesive cu End of Day.

1.2. Excluderi / non-obiective

- Conturi de client, înregistrare client cu email/parolă, istoricul clientului între vizite.
- Plăți reale, carduri, rambursări, plăți parțiale, fiscalizare, facturi fiscale.
- Tablete fizice și înrolare de dispozitive (Future Work); preview web în v1.
- Ceas virtual, simularea orelor, grafice orare și daily_occupancy_stats (respinse).
- Ocr automat pentru PDF scanate în v1; suportul pentru text neextractabil poate fi extins.
- Rezervări de jocuri cu expirare automată; nu expiră automat în v1.

2. Cerințe academice și trasabilitate

Cerințele de mai jos sunt preluate din README-ul repository-ului de referință al cursului și sunt tratate ca praguri minime ce trebuie demonstrate în aplicație. Tehnologiile exemplificate în repository (de exemplu Vue și MySQL) nu sunt obligatorii; sunt acceptate orice framework frontend și motor de bază de date, conform README.

Categorie	Cerință	Acoperire planificată
Bază de date	≥8 tabele relaționate, fiecare cu aproximativ 3–4 coloane sau mai mult	Model candidat: 21 Core + 2 AI; legătura joc–categorie are 2 coloane și nu este baza argumentului pentru prag.
Backend Java	≥12 endpoint-uri CRUD	Catalog de mese, produse, categorii, jocuri, exemplare, staff, configurări.
Backend complex	≥5 endpoint-uri complexe, ≥2 custom queries	Rezervare, checkout, plată, End of Day, filtrare jocuri și rapoarte.
Server activ	WebSocket / notificări	Actualizări comenzi, cereri joc, status masă, notă.
Frontend	≥10 funcții, ≥3 complexe	Fluxuri client, staff și manager, documentate în §14.

Categorie	Cerință	Acoperire planificată
Client mobil	≥3 operații	Scanare/intrare, filtrare jocuri/meniu, comandă/cerere; acceptarea PWA ca aplicație mobilă rămâne de confirmat.
AI	Model AI utilizat real	Re-ranking explicat, chatbot reguli PDF; fallback nu este prezentat ca AI.
Documentație	Diagramă bloc, use-case, secvență	Planificate și incluse în livrabilele de documentație.
Consistență	Fluxuri complete fără editări DB manuale	Bootstrap, CRUD și operații de workflow trebuie să facă datele utilizabile.
Bonus	Microservicii, deploy	Arhitectură Core + AI și opțiuni de publicare, de validat practic.

Sursă: https://github.com/cuibus/SCDproiect1, secțiunea „Project requirements". Confirmarea că un PWA satisface cerința de aplicație mobilă este o dependență externă nerezolvată.

3. Domeniu, actori și permisiuni

Actor	Autentificare	Capabilități
Vizitator	QR temporar + PIN + nume	Intră într-o sesiune; primește identitate de participant.
Participant	Token/cookie limitat la sesiune	Vede meniul/jocurile, coșul propriu, plasează comenzi, cere jocuri, vede membrii, își vede comenzile; poate declara persoane, iniția checkout și plăti nota comună.
Staff	Email + parolă	Deschide sesiuni, accesează preview-ul, procesează comenzi și jocuri, închide/returnează operațional și redeschide sesiuni înainte de plată.
Manager	Email + parolă, rol superior	Toate drepturile staff, plus CRUD admin, tarife, limită jocuri, PDF reguli, rapoarte, End of Day, export PDF.
AI Service	Apel intern din Core	Rerankează opțiuni eligibile și răspunde la întrebări despre reguli; nu decide starea financiară/operativă.



Figura 2. Cazuri de utilizare reprezentative pentru cei trei actori ai aplicației.

3.1. Limite de autorizare

- Un client nu are cont și nu poate executa operații staff/manager.

- Participantul poate modifica/anula numai propriile comenzi până la începerea acceptării; plata şi checkout-ul sunt comune sesiunii.
- La nivel de API, sesiunea efectiv autorizată derivă din identitatea participantului, nu dintr-un ID arbitrar trimis de frontend.
- Rolul MANAGER include drepturile STAFF; accesul la End of Day şi configurările administrative rămâne exclusiv managerului.
- Preview-ul QR/PIN este o rută protejată de staff; nu creează un „cont de tabletă”.

4. Fluxuri funcționale și reguli

4.1. Deschiderea mesei și intrarea participantului

1. Staff-ul alege o masă activă şi liberă şi deschide o sesiune asociată zilei simulate OPEN.
2. La deschidere se copiază tariful curent per persoană în sesiune, se generează un token QR şi un PIN noi; există cel mult o sesiune neînchisă pe masă.
3. Staff-ul deschide preview-ul per masă în interfaţa React Management. Preview-ul prezintă QR-ul şi PIN-ul curente într-un ecran de tip tabletă.
4. Clientul scanează codul cu iPhone-ul, accesează URL-ul HTTPS de intrare, introduce PIN-ul şi un display name.
5. Core creează SessionParticipant şi îi acordă acces strict la sesiunea respectivă; mai mulţi participanţi pot intra în aceeaşi sesiune.
6. La refresh, identificatorul de participare se poate restabili dacă tokenul persistă; numele nu este o cheie unică. Ieşirea participantului nu şterge obligaţiile istorice.
7. La închiderea mesei, tokenurile QR/PIN sunt invalidate; sesiunea nouă are credenţiale noi. Tabletele fizice rămân în Future Work.

4.2. Meniu, coșuri și comenzi

1. Clientul vede categoriile, produsele, preţurile şi disponibilitatea, îşi construieşte coşul individual. Stocarea locală a coşului în React este PROPUSĂ, nu încă îngheţată.
2. La submit, backend-ul verifică disponibilitatea şi preţul, apoi creează FoodOrder + OrderItem cu nume şi preţ snapshot.
3. Clientul poate edita/anula propria comandă numai până la începerea acceptării de către staff.
4. Staff-ul acceptă sau respinge fiecare poziţie; acceptarea într-un singur lot atomic este PROPUSĂ pentru a evita stările ambigue.
5. Dacă toate poziţiile sunt respinse, FoodOrder = CANCELLED şi total facturabil zero. Poziţiile respinse nu intră în notă.
6. Comenzile acceptate urmează ACCEPTED → PREPARING → READY → DELIVERED. Livrarea împreună a poziţiilor acceptate dintr-o comandă este PROPUSĂ pentru v1.
7. Nota comună se calculează din toate comenzile livrate şi acceptate, plus tarife; nu există „coş comun”.

4.3. Jocuri și inventar

1. Managerul administrează titlurile, categorii multiple, exemplare identificabile, dificultate, durata, vârsta, regulile scurte şi starea fizică.

2. Clientul filtrează după număr jucători, durată, vârstă, categorie, dificultate şi disponibilitate şi solicită un TITLU.
3. Staff-ul acceptă o cerere prin rezervarea atomică a unui exemplar USABLE fără alocare activă; cererea REQUESTED devine RESERVED.
4. Exemplarul RESERVED este indisponibil altor mese, dar nu ocupă limita de jocuri DELIVERED.
5. La livrare se reverifică atomic limita de jocuri DELIVERED per sesiune; la returnare cererea devine RETURNED şi copia este disponibilă sau intră în mentenanţă.
6. Clientul poate anula până la DELIVERED. Rezervările nu expiră automat; avertismentele pentru rezervări îndelungate sunt opţionale.
7. Înainte de plată, toate cererile trebuie să fie RETURNED, REJECTED sau CANCELLED; RESERVED/DELIVERED nu pot rămâne nerezolvate.

4.4. Tarife, checkout, redeschidere şi plată

1. Tariful monetar per persoană se copiază în TableSession la DESCHIDERE şi rămâne neschimbat chiar dacă managerul modifică tariful general.
2. Numărul de persoane fizic participante se declară manual de orice client, independent de numărul telefoanelor participante; include persoane care au plecat deja.
3. Clientul poate modifica numărul numai în ACTIVE înainte de PRIMUL checkout; dacă lipseşte ori este 0, checkout-ul solicită minimum 1 şi nu schimbă starea.
4. Primul checkout trece sesiunea în CHECKOUT_PENDING şi setează feesLocked=true permanent pentru clienţi. Nu se mai creează comenzi/cereri noi.
5. Staff/manager poate redeschide înainte de plata reuşită: status revine ACTIVE, însă feesLocked rămâne true. Staff poate corecta quantity după redeschidere, iar noul total se notifică.
6. Plata rămâne blocată până la finalizarea tuturor comenzilor şi cererilor de joc; un participant poate plăti integral fără aprobarea grupului ori a staff-ului.
7. Backend-ul recalculează totalul şi înregistrează maximum o plată simulată SUCCEEDED; în aceeaşi tranzacţie închide sesiunea şi eliberează masa.
8. Dacă plata simulată eşuează, sesiunea rămâne în CHECKOUT_PENDING pentru reîncercare (PROPUS).

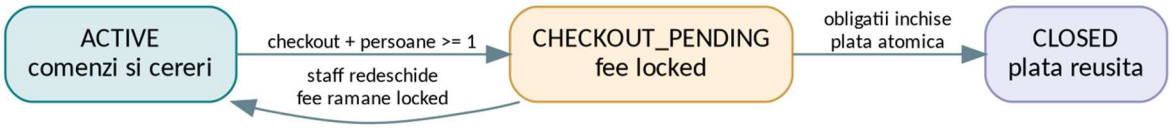


Figura 3. Tranziţiile principale ale unei sesiuni; „plata permisă” este o condiţie calculată.

4.5. End of Day şi simularea zilelor

1. Numai MANAGER poate apăsa End of Day; operaţia este refuzată până când TOATE sesiunile cafenelei sunt CLOSED.
2. Datele sunt asociate unui BusinessDay prin business_day_id la crearea sesiunii, nu prin data reală a laptopului.
3. End of Day calculează şi salvează un snapshot definitiv: venit total, venit meniu, venit tarife, sesiuni, produse, statistici per produs şi joc.
4. În aceeaşi tranzacţie, ziua curentă devine FINALIZED şi ziua următoare devine OPEN; repetarea aceleiaşi operaţii nu dublează raportul.

5. Dashboard-ul prezintă comparații pe ZILE simulate, grafice de venituri, produse și popularitatea jocurilor; managerul poate exporta PDF.
6. NU există ceas virtual, intervale orare simulate, grafice orare ori tabel daily_occupancy_stats. Programul real de funcționare este informativ.

5. Stări și invarianti

Agregat	Tranziții valide	Invariant / interpretare
TableSession	ACTIVE → CHECKOUT_PENDING → CLOSED; CHECKOUT_PENDING → ACTIVE numai prin staff înainte de plata reușită	După primul checkout feesLocked rămâne true.
FoodOrder	PLACED → ACCEPTED → PREPARING → READY → DELIVERED; PLACED → CANCELLED	CANCELLED când client anulează la timp sau toate pozițiile au fost respinse.
OrderItem	PENDING → ACCEPTED sau REJECTED	Decizia poziției nu trebuie confundată cu starea de preparare a comenzii.
GameRequest	REQUESTED → RESERVED → DELIVERED → RETURNED; REQUESTED → REJECTED/CANCELLED; RESERVED → CANCELLED	RESERVED și DELIVERED țin inventarul; numai DELIVERED ocupă limita mesei.
Payment	PENDING → SUCCEEDED sau FAILED	Maximum una SUCCEEDED per sesiune.
BusinessDay	OPEN → FINALIZED	Maximum o zi OPEN; raport final unic.

5.1. Invarianti globali

- Maximum o sesiune neînchisă per masă; maximum o zi OPEN; maximum o plată SUCCEEDED per sesiune.
- Un exemplar de joc nu poate fi legat de două cereri RESERVED/DELIVERED active; alocarea trebuie să corespundă titlului solicitat.
- Orice participant dintr-un FoodOrder, GameRequest sau Payment trebuie să aparțină aceleiași TableSession.
- O sesiune CLOSED are obligațiile rezolvate, plata confirmată și accesul QR/PIN invalidat.
- Un raport FINALIZED nu se modifică prin editarea ulterioară a catalogului sau a meniului.
- Evenimentele WebSocket nu anunță succesul înainte de confirmarea tranzacției PostgreSQL.

6. Arhitectură și comunicare

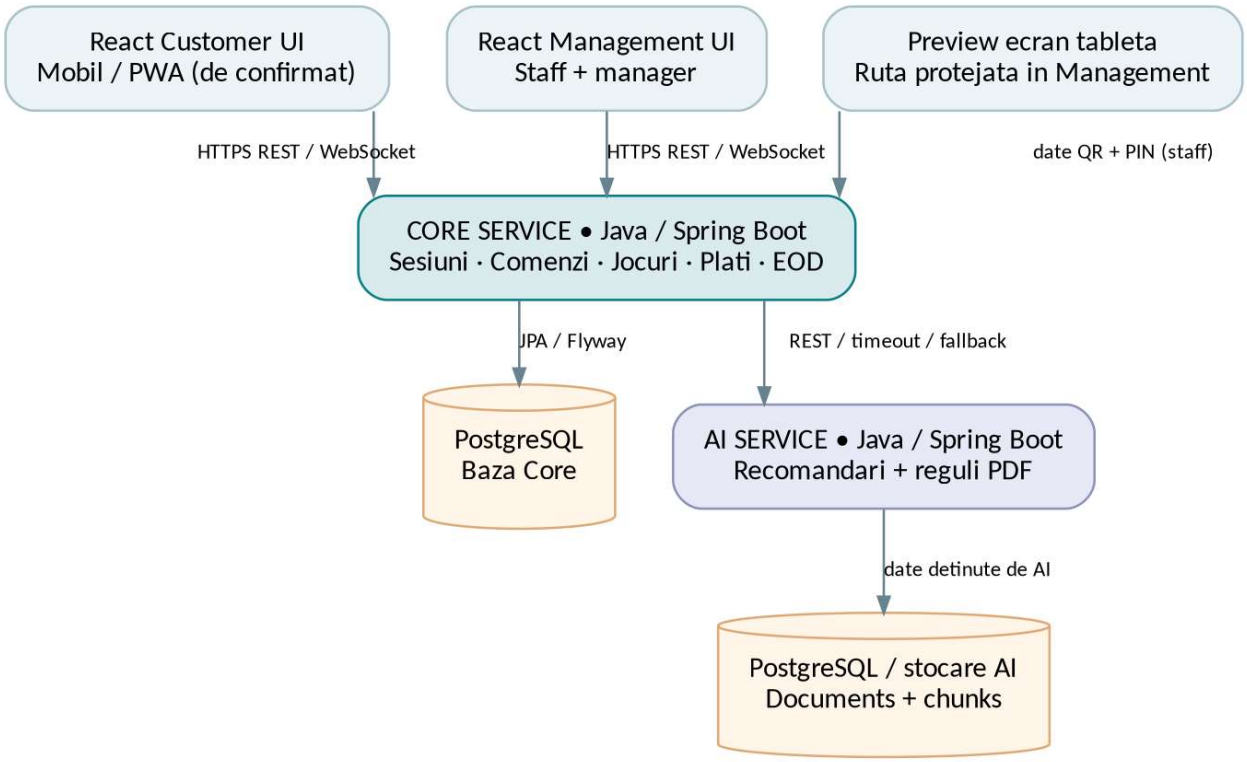


Figura 4. Arhitectură logică, două servicii Java şi interfeţe React.

6.1. Componente

Componentă	Responsabilităţi	Statut
React Customer UI	Intrare QR/PIN, catalog, coş, comenzi, cereri joc, tarif, checkout, plată.	CONFIRMAT ca funcţii; PWA mobil ca formă de livrare = DESCHIS academic.
React Management UI	Staff/manager, preview QR/PIN per masă, procesare, configurări, End of Day.	CONFIRMAT
Core Service (Spring Boot)	Sursa de adevăr pentru operaţii, autorizare, tranzacţii, inventar, facturare, rapoarte.	CONFIRMAT ca responsabilităţi; împărţirea în microservicii = propunere favorită.
AI Service (Spring Boot)	Reranking + explicaţii şi asistent PDF prin prelucrare/retrieval.	PROPUS ca serviciu independent.
PostgreSQL Core / AI	Persistenţă relaţională, integritate, tranzacţii, document chunks.	PROPUS / PREFERAT
REST + WebSocket	Comenzi şi recuperare stare prin REST; notificări server active prin WebSocket.	CONFIRMAT ca mecanisme.

6.2. Comunicare şi degradare controlată

- React → Core: HTTPS/JSON pentru operaţii; WebSocket autentificat pentru notificări de stare. După reconectare, REST restabileşte starea autoritară.
- Core → AI: REST, timeout, erori tratate. Dacă AI pentru recomandări nu răspunde, Core returnează lista deterministă filtrată, fără explicaţie AI.
- AI nu face scrieri directe în tabelele Core şi nu decide eligibilitatea operaţională sau suma notei.

- Service AI deține metadatele și fragmentele PDF; game_id este referință logică între servicii, nu FK cross-database.
- Preview-ul QR/PIN este rută protejată în Management; nu există aplicație sau tabel pentru tablete fizice în v1.

7. Infrastructură și deploy

Strat	Alegere curentă	Motiv / condiții
Platformă dezvoltare	Windows + Git + Java JDK + Maven + Node/npm + IDE	Mediu local cunoscut; fără tooluri care cer hardware special.
Backend	Java, Spring Boot, Spring Web, Spring Data JPA, Spring Security, WebSocket	Aliniere cu cerința Java și cu obiectivul didactic.
Frontend	React (client și management), UI responsive	Separare roluri și reutilizarea cunoștințelor JavaScript.
Bază de date	PostgreSQL preferat	Chei externe, tranzacții și indexuri unice parțiale.
Versionare DB	Code First JPA/Hibernate + Flyway după stabilizare	Entități în Java; migrații controlate la deploy; Hibernate validate în medii persistente.
Ambalare	Docker / Docker Compose propuse	Pornire reproductibilă pentru backend-uri și DB; bonus de infrastructură.
Deploy	Hosting cu HTTPS, DB persistentă și variabile secrete	Furnizor, plan, cost și topologie rămân DESCHISE.
Fișiere PDF	Stocare fișiere persistentă, pointer storage_key în AI DB	Local pentru dezvoltare; locația finală de stocare = DESCHISĂ.
Observabilitate	Loguri structurate fără secrete, health checks, backup DB	PROPUS; minimul trebuie adaptat bugetului.

7.1. Medii și date

- LOCAL: laptop Windows; serviciile pot rula prin IDE și PostgreSQL local ori Docker.
- DEMO/DEPLOY: acces HTTPS public și mobil; QR nu trebuie să conțină localhost când este scanat cu iPhone-ul.
- CONFIG: URL public, credențiale DB, secret de criptare, tokenuri și cheia modelului AI prin variabile de mediu / secret management.
- PERSISTENȚĂ: volume sau DB gestionată; fișierele PDF nu trebuie pierdute la redeploy.
- COST: alegerea furnizorilor se face ulterior; nu este fixat un buget ori un provider.

7.2. Opțiuni nefinalizate

Nu sunt confirmate: providerul de hosting, varianta gratuită/plătită, modul de rulare AI (API extern sau model local), domeniul final, gestionarea secretelor și backup-ul efectiv. Acestea vor fi validate într-un ADR de infrastructură înainte de deploy.

8. Model de date: inventar și relații

Schema candidată după eliminarea tabletelor fizice și a rapoartelor orare conține 21 de tabele pentru Core Service și două pentru AI Service. Game–Category este N:M prin game_category_links; celelalte relații de bază sunt 1:N sau 1:0..1.

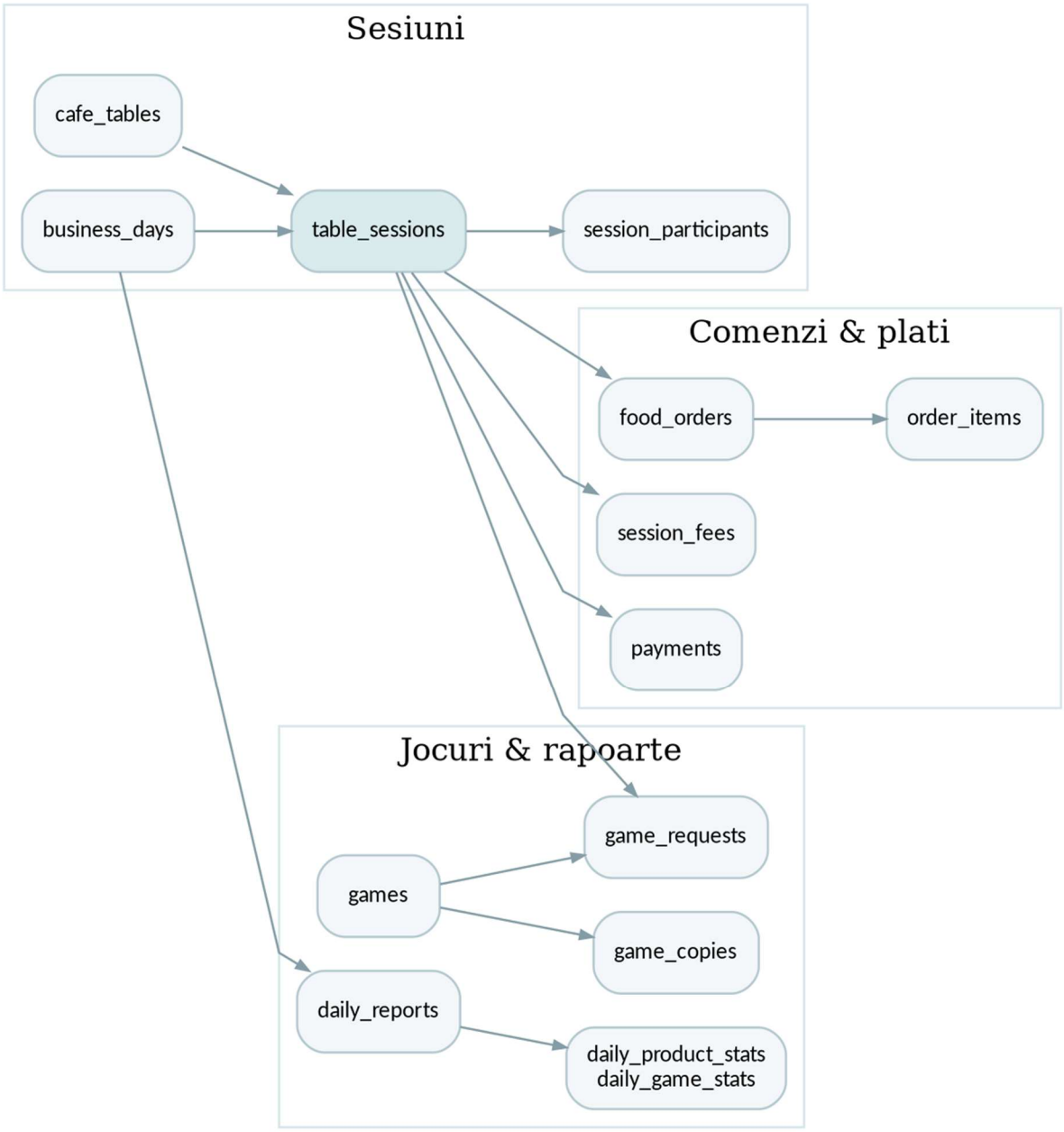


Figura 5. Relații conceptuale între principalele agregate (schemă simplificată).

Domeniu	Tabele Core
Securitate	staff_users
Mese și sesiuni	cafe_tables, table_sessions, session_participants
Jocuri	games, game_categories, game_category_links, game_copies, game_requests
Meniu și comenzi	menu_categories, menu_items, food_orders, order_items
Facturare	session_fees, payments
Setări	cafe_settings, opening_hours
Zile și rapoarte	business_days, daily_reports, daily_product_stats, daily_game_stats

AI Service: rules_documents și rules_chunks. table_devices, daily_occupancy_stats, shopping_carts, cart_items și bills NU intră în schema de bază curentă.

8.1. Principii de modelare

- PK propus BIGINT/Long generat; FK explicite; PostgreSQL DECIMAL/NUMERIC pentru sume și BigDecimal în Java.
- Enum-uri persistate textual; nume de clase Java CamelCase și nume de tabele snake_case.
- Soft-deactivation pentru jocuri, produse, categorii și staff cu istoric; nu se șterg în cascadă tranzacțiile istorice.
- Prețuri și nume snapshot în pozițiile de comandă; tariful de acces snapshot în TableSession la deschidere.
- Disponibilitatea mesei și exemplarelor este derivată din înregistrările operaționale, nu din contoare redundante.
- Snapshot-ul zilnic este intenționat denormalizat pentru raportare istorică nemodificabilă.

9. Dicționar complet de date

Notație: PK = cheie primară; FK = cheie externă; UQ = unicitate; NN = NOT NULL; NULL = permis; „prop.” = câmp sau regulă propusă, nu înghețată. Listele reprezintă designul candidat pentru Code First, nu o migrare SQL executată.

9.1. staff_users

Conturi staff și manager; utilizatorii clienți nu au conturi.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK, generat
fullName	String	NN, max 100
email	String	NN, UQ, max 255
passwordHash	String	NN; niciodată parola în clar
role	StaffRole	NN: STAFF / MANAGER
active	Boolean	NN, implicit true
createdAt	Instant	NN

Regulă: Nu se șterge fizic un angajat referit de sesiuni/rapoarte. Autorizarea se face server-side.

9.2. cafe_tables

Mese fizice administrate, fără isOccupied persistent.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
tableNumber	Integer	NN, UQ, >0
capacity	Integer	NN, >0
active	Boolean	NN, implicit true
createdAt	Instant	NN

Regulă: O masă are 0..N sesiuni istorice, dar maximum una cu status diferit de CLOSED.

9.3. table_sessions

Agregat central: ciclul mesei, tarif fix, QR/PIN și concurență.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
table	CafeTable	FK, NN
businessDay	BusinessDay	FK, NN
openedBy	StaffUser	FK, NN
status	SessionStatus	NN: ACTIVE/CHECKOUT_PENDING/CLOSED
accessFeeUnitPrice	BigDecimal	NN, >=0; fix la deschidere
feesLocked	Boolean	NN; true după primul checkout
joinTokenDigest	String	NN/UQ; prop. hash token QR
joinTokenEncrypted	String	NN în timp activ; prop. afișaj securizat
pinHash	String	NN; prop. verificare PIN
pinEncrypted	String	NN în timp activ; prop. afișaj securizat
credentialsCreatedAt	Instant	NN, prop.
version	Long	NN; optimistic locking
startedAt	Instant	NN
closedAt	Instant	NULL până la CLOSED

Regulă: Index unic parțial pe table_id unde status <> CLOSED. Credențiale QR/PIN se invalidează la CLOSED; metoda criptografică rămâne DESCHISĂ.

9.4. session_participants

Participanți temporari; fiecare poate comanda independent.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
session	TableSession	FK, NN
displayName	String	NN, max 80; nu este unic
accessTokenHash	String	NN, UQ
active	Boolean	NN
joinedAt	Instant	NN
leftAt	Instant	NULL cât este activ

Regulă: La ieșire se revocă accesul, fără a șterge obligațiile. Reconnect cu token valid refolosește același participant.

9.5. game_categories

Categorii de joc disponibile pentru filtrare.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
name	String	NN, UQ
description	String	NULL
active	Boolean	NN

Regulă: Un joc poate avea mai multe categorii; minimum o categorie activă pentru joc publicat este PROPUS.

9.6. games

Titluri de joc; nu reprezintă exemplare fizice.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
title	String	NN, max 150
description	String	NN / text
shortRules	String	PROPUS: rezumat reguli, text
minPlayers	Integer	NN, >=1
maxPlayers	Integer	NN, >= minPlayers
durationMinutes	Integer	NN, >0
minimumAge	Integer	NN, >=0
difficulty	GameDifficulty	NN: EASY/MEDIUM/HARD
active	Boolean	NN
createdAt	Instant	NN

Regulă: Nu are category_id singular și nu stochează availableCopies. Disponibilitatea derivă din copii și rezervări.

9.7. game_category_links

Tabel de legătură JPA many-to-many, fără @Entity obligatorie.

Câmp / relație	Tip Java	Restricție / explicație
game_id	Long	PK compus + FK games
category_id	Long	PK compus + FK game_categories

Regulă: Cele două coloane sunt suficiente tehnic; nu se numără individual în minimul academic de 3–4 coloane.

9.8. game_copies

Exemplare fizice individuale.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK

Câmp / relație	Tip Java	Restricție / explicație
game	Game	FK, NN
inventoryCode	String	NN, UQ
conditionStatus	CopyCondition	NN: USABLE/UNDER_MAINTENANCE/UNAVAILABLE
notes	String	NULL
createdAt	Instant	NN

Regulă: Nu stocăm RESERVED/DELIVERED ca stare fizică; acestea sunt stări ale cererii.

9.9. game_requests

Solicitări + istoric împrumuturi.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
session	TableSession	FK, NN
participant	SessionParticipant	FK, NN
game	Game	FK, NN
gameCopy	GameCopy	FK, NULL până la acceptare
status	GameRequestStatus	NN
requestedAt	Instant	NN
reservedAt	Instant	NULL
deliveredAt	Instant	NULL
returnedAt	Instant	NULL
cancelledAt	Instant	NULL
processedBy	StaffUser	FK, NULL
resolutionReason	String	NULL
version	Long	PROPUS pentru concurență

Regulă: Unicitate condiționată a game_copy_id doar în RESERVED/DELIVERED; copy.game_id trebuie să coincidă cu request.game_id.

9.10. menu_categories

Grupează produse de meniu.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
name	String	NN, UQ
description	String	NULL
active	Boolean	NN

Regulă: Fiecare MenuItem are o singură categorie (1:N), distinct de categoriile N:M ale jocurilor.

9.11. menu_items

Produse și prețuri curente.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
category	MenuCategory	FK, NN
name	String	NN
description	String	NULL
price	BigDecimal	NN, >=0
available	Boolean	NN
active	Boolean	NN
createdAt	Instant	NN

Regulă: Produs comandabil numai dacă active și available sunt true. Modificarea prețului nu afectează comenzile vechi.

9.12. food_orders

Comenzi ale unui participant într-o sesiune.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
session	TableSession	FK, NN
participant	SessionParticipant	FK, NN
status	OrderStatus	NN
cancellationReason	OrderCancellationReason	NULL
processedBy	StaffUser	FK, NULL
placedAt	Instant	NN
acceptedAt	Instant	NULL
readyAt	Instant	NULL
deliveredAt	Instant	NULL
version	Long	NN/prop. optimistic locking

Regulă: Cerința de minimum o poziție și apartenența participantului la sesiune se validează tranzacțional.

9.13. order_items

Poziții de comandă cu preț și nume snapshot.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
order	FoodOrder	FK, NN
menuItem	MenuItem	FK, NN

Câmp / relație	Tip Java	Restricție / explicație
productNameSnapshot	String	NN
quantity	Integer	NN, >0
unitPrice	BigDecimal	NN, >=0
status	OrderItemStatus	NN: PENDING/ACCEPTED/REJECTED
rejectionReason	String	NULL

Regulă: Numai pozițiile ACCEPTED din comenzi DELIVERED sunt facturabile. Rejected și CANCELLED nu intră în total.

9.14. session_fees

Număr fizic de persoane tarifate într-o sesiune.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
session	TableSession	FK, NN, UQ
quantity	Integer	NN, >=1 dacă înregistrarea există
version	Long	NN/prop.
updatedAt	Instant	NN
updatedByParticipant	SessionParticipant	FK, NULL, PROPUS
updatedByStaff	StaffUser	FK, NULL, PROPUS

Regulă: 0..1 per sesiune. Prețul nu se duplică aici; TableSession.accessFeeUnitPrice este snapshot-ul fix.

9.15. payments

Încercări de plată integrală simulată.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
session	TableSession	FK, NN
initiatedBy	SessionParticipant	FK, NN
amount	BigDecimal	NN, >=0
status	PaymentStatus	NN: PENDING/SUCCEEDED/FAILED
method	PaymentMethod	NN: SIMULATED
idempotencyKey	UUID	NN, UQ
createdAt	Instant	NN
completedAt	Instant	NULL
failureReason	String	NULL

Regulă: Index unic parțial pe session_id doar pentru SUCCEEDED. Salvarea reușitei și CLOSED într-o tranzacție.

9.16. cafe_settings

Configurații globale administrate de manager.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK; un singur rând global, prop.
cafeName	String	NN
accessFeePrice	BigDecimal	NN, >=0; numai pentru sesiuni noi
maxDeliveredGamesPerTable	Integer	NN, >0
updatedAt	Instant	NN

Regulă: Limita se aplică la livrări viitoare; nu revocă automat împrumuturi deja DELIVERED (PROPUS).

9.17. opening_hours

Program informativ al cafenelei, fără legătură cu EOD.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
dayOfWeek	Integer	NN, 1..7, UQ prop.
openTime	LocalTime	NULL
closeTime	LocalTime	NULL
closed	Boolean	NN

Regulă: Nu restricționează deschiderea meselor și nu determină ziua simulată.

9.18. business_days

Zi simulată, diferită de ziua calendaristică.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
dayNumber	Integer	NN, UQ, >0
label	String	NN
status	BusinessDayStatus	NN: OPEN/FINALIZED
openedAt	Instant	NN: timp real pentru audit
finalizedAt	Instant	NULL
finalizedBy	StaffUser	FK, NULL
version	Long	PROPUS pentru concurență

Regulă: Cel mult un BusinessDay OPEN. NO virtual minute, NO hour-based statistics. Ziua nouă apare atomic cu finalizarea.

9.19. daily_reports

Snapshot financiar și operațional per zi.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
businessDay	BusinessDay	FK, NN, UQ
totalRevenue	BigDecimal	NN, >=0
menuRevenue	BigDecimal	NN, >=0
accessFeeRevenue	BigDecimal	NN, >=0
closedSessionsCount	Integer	NN, >=0
deliveredOrdersCount	Integer	PROPUS; NN, >=0
productsSoldCount	Integer	NN, >=0
gamesDeliveredCount	Integer	NN, >=0
generatedAt	Instant	NN

Regulă: TotalRevenue = menuRevenue + accessFeeRevenue pentru v1 fără alte taxe. Snapshot-ul FINALIZED rămâne nemodificabil.

9.20. daily_product_stats

Statistică pe produs și zi simulată.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
report	DailyReport	FK, NN
menuItem	MenuItem	FK, NN
productNameSnapshot	String	NN
quantitySold	Integer	NN, >=0
revenue	BigDecimal	NN, >=0

Regulă: UQ(report_id, menu_item_id). Poate exista o singură agregare cu prețuri istorice diferite pentru același produs.

9.21. daily_game_stats

Statistică pe titlu de joc și zi simulată.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
report	DailyReport	FK, NN
game	Game	FK, NN
gameTitleSnapshot	String	NN
timesRequested	Integer	NN, >=0
timesDelivered	Integer	NN, >=0

Regulă: UQ(report_id, game_id). Jocuri populare = împrumuturi efective livrate și, la finalizarea zilei, returnate.

9.22. rules_documents [AI]

Document PDF de reguli deținut de serviciul AI.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
gameId	Long	Referință logică Core, NU FK între DB-uri
originalFilename	String	NN
storageKey	String	NN
processingStatus	DocumentStatus	UPLOADED/PROCESSING/READY/FAILED
uploadedAt	Instant	NN

Regulă: Înlocuirea PDF trebuie să activeze versiunea nouă numai după procesare reușită. Metadate versiune/staged pot fi necesare; DESCHIS.

9.23. rules_chunks [AI]

Fragmente text din PDF pentru căutarea informațiilor.

Câmp / relație	Tip Java	Restricție / explicație
id	Long	PK
document	RulesDocument	FK, NN
pageNumber	Integer	NN, >=1
chunkIndex	Integer	NN, >=0
content	String	NN / TEXT

Regulă: UQ(document_id, chunk_index) PROPUS. Citații pe pagini când sursa permite; OCR în afara v1.

9.24. Indexuri și constrângeri esențiale

Tabel	Integritate PostgreSQL	Justificare
cafe_tables	UQ(table_number)	Identitate unică a mesei
table_sessions	UQ condiționat(table_id, status <> CLOSED)	Maximum o sesiune deschisă/neînchisă per masă
game_copies	UQ(inventory_code)	Identificarea exemplarului fizic
game_category_links	PK/UQ(game_id, category_id)	Nicio asociere duplicată
game_requests	UQ condiționat(game_copy_id, status în RESERVED/DELIVERED)	Fără rezervări/livrări concurente ale aceleiași copii
session_fees	UQ(session_id)	O singură cantitate per sesiune
payments	UQ(idempotency_key) + UQ condiționat(session_id, SUCCEEDED)	Reîncercare sigură și o singură plată reușită
business_days	UQ(day_number) + UQ condiționat(status OPEN)	Zile simulate ordonate, o singură zi deschisă
daily_reports	UQ(business_day_id)	Un snapshot final pentru fiecare zi
daily_product_stats	UQ(report_id, menu_item_id)	Fără statistici duplicat
daily_game_stats	UQ(report_id, game_id)	Fără statistici duplicat

9.25. Cardinalități și relații JPA

Relație	Cardinalitate	Interpretare
CafeTable – TableSession	1:N	@ManyToOne pe TableSession; index unic condiționat pentru neînchise.
BusinessDay – TableSession	1:N	@ManyToOne pe TableSession; toate tranzacțiile zilei moștenesc business day.
TableSession – SessionParticipant	1:N	@ManyToOne pe participant; identitate fără cont.
Game – GameCategory	N:M	@ManyToMany + @JoinTable game_category_links.
Game – GameCopy	1:N	@ManyToOne pe GameCopy.
GameCopy – GameRequest	1:N istoric, 0..1 activ	Nullable la REQUESTED; rezervare atomică.
TableSession / Participant – FoodOrder	1:N	Două FK; verificare de apartenență la aceeași sesiune.
FoodOrder – OrderItem	1:N	Compoziție logică; fără ștergeri periculoase după plasare.
TableSession – SessionFee	1:0..1	@OneToOne + FK UNIQUE.
TableSession – Payment	1:N încercări	@ManyToOne din Payment; index pentru unicul SUCCEEDED.
BusinessDay – DailyReport	1:0..1	@OneToOne + FK UNIQUE.
DailyReport – Product/GameStat	1:N	@ManyToOne pe fiecare rând de statistică.
RulesDocument – RulesChunk	1:N în AI	@ManyToOne pe chunk.

10. JPA, Code First și migrații

10.1. Politica de modelare

- @Entity pentru tabelele operaționale; game_category_links poate fi administrat fără clasă Java proprie.
- @ManyToOne ca legătură principală de navigare; relații bidirecționale doar unde sunt utile (de exemplu FoodOrder + OrderItem).
- LAZY explicit pentru asocieri; DTO-uri pentru API, nu expunerea entităților JPA direct prin JSON.
- Enum-uri stocate ca STRING, nu ordinal; BIGDECIMAL pentru bani; Instant pentru momente reale de audit.
- @Version pe agregatele cu modificări concurente: sesiuni, comenzi, cereri joc, tarife, posibil zile.
- Fără CascadeType.REMOVE de la Game, MenuItem, TableSession către istoric. Deactivare în loc de delete.
- Controller → application service → domain rules → repository; DTO-uri și validări de intrare clare.

10.2. Evoluția schemei

Etapă	Mod de lucru	Protecție
Model inițial local	Entitățile definesc structura; Hibernate poate genera temporar schema pentru explorare.	Date demonstrative regenerabile, fără dependență de schema auto-update.
Schema stabilizată	Flyway înregistrează migrațiile SQL și indexurile speciale.	Hibernate rulează validate, nu modifică schema persistentă.
Deploy / producție demo	Migrații versionate, date persistente, backup și rollback planificat.	Fără ddl-auto=create sau auto-update în mediu persistent.

Cerința „Code First” nu interzice SQL: indexurile unice parțiale și anumite CHECK-uri compuse necesită migrații explicite PostgreSQL. Strategia Code First + Flyway este propusă ca abordare finală.

11. Securitate și identitate

11.1. Staff și RBAC

- Email unic, parole numai sub formă de hash adaptat parolelor; STAFF și MANAGER ca roluri în backend.
- Autorizare în Core Service pentru fiecare endpoint; UI ascunde acțiuni, dar nu este sursa de adevăr.
- Preview-ul QR/PIN și API-ul lui sunt disponibile doar staff/manager autentificat.
- Managementul inițial al primului cont manager necesită un bootstrap sigur, fără editare manuală DB.

11.2. QR/PIN și participare

Subiect	Decizie / regulă	Stare
QR sesiune	Token de intrare nou, aleatoriu, per TableSession; link HTTPS, nu doar tableNumber.	CONFIRMAT + DERIVAT
PIN sesiune	Cod nou per sesiune, introdus după scanare; 6 cifre este recomandare, nu încă obligatoriu.	CONFIRMAT / PROPUS
Preview	Management React, poate fi reafișat pe laptop în mod fullscreen; aceeași pereche QR/PIN pe sesiune.	CONFIRMAT
Persistență	Digest/hash pentru validare și criptare reversibilă pentru reafișare sunt propuse; chei în secrete de mediu.	PROPUS
Checkout	Noi participanți blocați și codurile ascunse în CHECKOUT_PENDING este PROPUS; după redeschidere se reafișează aceleași coduri.	PROPUS
Close	Tokenurile de intrare devin invalide și recuperarea secretelor se închide.	CONFIRMAT
Participare	Token distinct pentru SessionParticipant; refresh nu recreează persoana când tokenul e valid.	CONFIRMAT / DERIVAT
Client mobil	QR trebuie să conțină URL accesibil pe iPhone, niciodată localhost la demo.	DERIVAT

11.3. Măsuri suplimentare

- Limitare încercări PIN / rate limiting și protecții anti-abuz care nu permit blocarea permanentă a întregii mese.
- HTTPS, politici CORS adecvate, protecții XSS/CSRF compatibile cu modul ales de stocare a sesiunii.
- Nu se loghează parole, QR token, PIN, token participant, secrete de criptare sau date sensibile ale cererilor.
- WebSocket autentificat și autorizat pe sesiune; fără difuzarea PIN/QR în broadcast.
- API-ul respinge sesiuni client arbitrare, verifică participant.sessionId pentru comenzi, cereri și plată.
- Stocare securizată a tokenului de participant prin cookie HttpOnly este PROPUSĂ; mecanismul final rămâne deschis.

12. Tranzacții și consistență

Tranzacție	Secvență logică	Protecție
T1 • Accept game	Validează REQUESTED + sesiune; identifică și alocă atomic copy disponibilă; trece în RESERVED.	Index unic parțial + coordonare copie/cerere.
T2 • Deliver game	Pe sesiune, numără DELIVERED și verifică limita; trece RESERVED → DELIVERED.	Coordonare sesiunii pentru două livrări simultane.
T3 • Checkout	Pe sesiune, verifică quantity >= 1, trece în CHECKOUT_PENDING, blochează fee și noile comenzi.	Plasarea comenzii și checkout-ul se serializează logic.

Tranzacție	Secvență logică	Protecție
T4 • Payment	Verifică obligații, recalculează totalul și înregistrează unicul SUCCEEDED + CLOSED.	Idempotency + index unic + tranzacție atomică.
T5 • Reopen	Staff readuce CHECKOUT_PENDING la ACTIVE, fără a debloca fee pentru client.	Coordonare cu T4 pe aceeași sesiune.
T6 • End of Day	Asigură zero sesiuni neînchise, salvează toate statisticile, FINALIZED + următoarea OPEN.	Coordonare cu deschiderea sesiunilor; raport unic.
T7 • Process order	Decizie pe poziții, rollback la eroare și tranziție coerentă la ACCEPTED/CANCELLED.	@Version/lock și tranzacție.
T8 • Return game	Trezește disponibilitatea copiei și încheie împrumutul; opțional marchează mentenanță.	Tranziție unică, audit și notificare after commit.

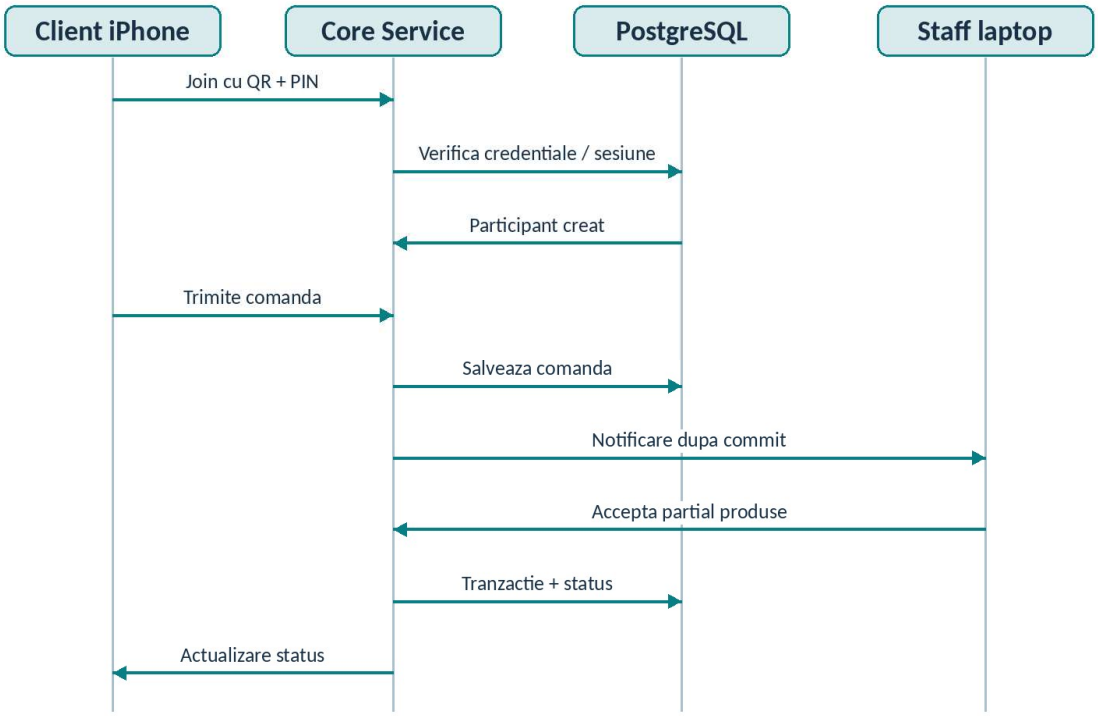


Figura 6. Diagramă de secvență conceptuală: autentificare client, comandă, procesare și actualizare după commit.

12.1. Principii de concurență

- Blocări consecvente pentru aceeași sesiune în fluxurile sensibile; inventarul poate necesita o blocare suplimentară pe exemplar.
- Un index unic condiționat garantează integritatea chiar dacă două request-uri trec ambele o verificare inițială.
- Optimistic locking (@Version) este un instrument de detectare, nu un substitut pentru coordonarea rezervării ori limitei.
- End of Day și „open session” trebuie să partajeze un mecanism tranzacțional care previne sesiuni create pe o zi finalizată.
- Notificările WebSocket sunt publicate după commit; dacă s-a pierdut evenimentul, clientul reface starea prin REST.
- Interogările complexe pentru raportare citesc datele zilei consistente; nu se calculează sume din inputul trimis de React.

13. API și WebSocket planificate

Lista este orientativă pentru proiectarea contractelor, nu un API înghețat. Prefixele /api/admin și /api/participant sugerează limitele de autorizare. Aceste exemple vor fi rafinate într-un catalog de endpoint-uri înainte de implementare.

13.1. Exemple de CRUD (minim 12 endpoint-uri)

Endpoint candidat	Rol
GET /api/admin/tables	Listă mese
POST /api/admin/tables	Creează masă
PATCH /api/admin/tables/{id}	Editează/activează masă
GET /api/admin/games	Listă jocuri
POST /api/admin/games	Creează titlu
PATCH /api/admin/games/{id}	Editează titlu
POST /api/admin/game-copies	Adaugă exemplar
PATCH /api/admin/game-copies/{id}	Actualizează stare exemplar
GET /api/admin/menu-items	Listă produse
POST /api/admin/menu-items	Creează produs
PATCH /api/admin/menu-items/{id}	Editează produs
POST /api/admin/menu-categories	Creează categorie meniu
POST /api/admin/game-categories	Creează categorie joc
GET /api/admin/staff	Listă staff
POST /api/admin/staff	Creează staff
PATCH /api/admin/staff/{id}	Activează/dezactivează angajat
PUT /api/admin/opening-hours/{day}	Editează program informativ

13.2. Operații complexe și custom queries

Endpoint candidat	Responsabilitate	Demonstrează
POST /api/admin/tables/{id}/sessions	Deschide sesiune, snapshot fee, QR/PIN, unicitate masă	Tranzacție
GET /api/participant/games/search	Filtre jucători/durată/dificultate/categorie + copii disponibile	CUSTOM QUERY 1
POST /api/admin/game-requests/{id}/reserve	Alocă exemplar disponibil atomic	Tranzacție
POST /api/admin/game-requests/{id}/deliver	Verifică limita DELIVERED	Tranzacție
POST /api/participant/sessions/current/checkout	Blochează tarife și operații noi	Tranzacție
POST /api/participant/sessions/current/pay	Notă autoritară, idempotency, SUCCEEDED + CLOSED	Tranzacție
POST /api/admin/business-days/{id}/finalize	Snapshot complet + next OPEN	Tranzacție
GET /api/admin/analytics/days/compare	Compară snapshot-uri, produse și jocuri între zile	CUSTOM QUERY 2
POST /api/admin/games/{id}/rules	Încarcă PDF prin Core către AI	Integrare
POST /api/participant/ai/recommend	Filtru determinist + AI rerank / fallback	Integrare

13.3. Evenimente WebSocket

- order.placed / order.status-changed / order.item-rejected
- game.requested / game.reserved / game.delivered / game.returned
- session.checkout-pending / session.reopened / session.closed / session.fee-updated
- business-day.finalized / business-day.opened pentru dashboard manager
- Evenimentele trimit ID-uri, statusuri și totaluri permise; niciodată token QR ori PIN.

14. Frontend și client mobil

14.1. Suprafața React

Interfață	Funcționalități planificate
Customer (mobil-first)	QR/PIN join, alegere nume, menu view/filter, cart, submit/edit/cancel order, order status, game catalog filters, game requests/cancel, member list, fees quantity, shared bill, checkout, simulated pay, AI recommendations și reguli PDF.
Management Staff	Login, overview mese, deschide sesiune, preview tabletă, comenzi cu acceptare/reject individual, stări preparare/livrare, cereri/reservări/livrări/returnări, redeschidere, corecție fee.
Management Manager	CRUD staff, mese, jocuri/categorii/copies, meniuri/categorii, settings, opening hours, upload rules PDF, dashboard zile, End of Day, export PDF.

14.2. Funcționalități frontend complexe demonstrabile

- Checkout + notă comună cu obligații, rate locked și plată integrală.
- Dashboard live pentru comenzile mai multor participanți și ale staff-ului.
- Inventar de jocuri cu filtre compuse și disponibilitate a exemplarelor.
- Admin flux de acceptare parțială a pozițiilor din comandă.
- Dashboard cu comparații între zile simulate, produse și jocuri.
- Recomandare AI pe criterii structurate, cu fallback fără AI.

14.3. Cerința de client mobil

Intenția este o interfață customer optimizată pentru iPhone, eventual instalabilă ca PWA. Acordul profesorului că aceasta îndeplinește cerința de „aplicație mobile" NU este confirmat. Dacă nu este acceptată, va trebui construit un client mobil dedicat ori o altă variantă explicit acceptată, cu minimum trei operații. Nu afirmăm echivalența dintre PWA și aplicație mobilă nativă.

15. Serviciul AI și documentele PDF

15.1. Recomandări bazate pe inventar

1. Clientul selectează structurat numărul jucătorilor, durata, dificultatea și categoria; Core calculează lista ELIGIBILĂ din catalog și inventar.
2. AI Service primește doar opțiunile eligibile și criteriile, apoi le rerankează/explică; AI nu poate „inventa" jocuri indisponibile.
3. Dacă AI este indisponibil, Core returnează rezultatele filtrării deterministe și mesajul „AI momentan indisponibil"; fără explicații pretinse AI.
4. Modelul AI trebuie invocat efectiv pentru recomandările explicate, nu înlocuit de o interogare SQL prezentată ca AI.

15.2. Asistent pentru regulile jocurilor

1. Managerul încarcă un PDF de reguli pentru un joc; v1 presupune PDF cu text extractibil (fără OCR).
2. AI Service extrage textul, îl fragmentează în rules_chunks și păstrează pagina pentru răspunsuri ancorate.
3. La întrebare, caută fragmente relevante, cere modelului să răspundă pe baza lor și să exprime incertitudinea când nu există dovadă.
4. Înlocuirea PDF trebuie să mențină vechiul document utilizabil până la procesarea reușită a celui nou.
5. Stocarea fișierelor, modelul AI, strategia de retrieval/embeddings și politica exactă de comutare sunt încă de stabilit.

15.3. Delimitarea microserviciilor

Core deține jocurile și toate stările operaționale. AI deține documentele, fragmentele și logica modelului. Nu se definesc FK între baze independente; se folosesc gameId și validări prin API. Indisponibilitatea AI nu trebuie să oprească comenzile, checkout-ul sau procesarea jocurilor.

16. End of Day și analitice

16.1. Indicatori finali

Indicator zilnic	Surse Core	Snapshot
Total încasări	payments.status = SUCCEEDED	daily_reports.total_revenue
Venit produse	order_items ACCEPTED din food_orders DELIVERED, pe sesiuni plătite	daily_reports.menu_revenue / daily_product_stats
Venit tarife	session_fees.quantity × table_sessions.accessFeeUnitPrice	daily_reports.access_fee_revenue
Sesiuni	table_sessions închise în business day	daily_reports.closedSessionsCount
Comenzi livrate	food_orders DELIVERED	daily_reports.deliveredOrdersCount (prop.)
Produse vândute	suma quantity din poziții facturabile	daily_product_stats.quantitySold
Cereri joc	game_requests create în ziua sesiunii	daily_game_stats.timesRequested
Împrumuturi efective	game_requests cu livrare + returnare	daily_game_stats.timesDelivered

16.2. Zile simulate, nu ore simulate

Ziua business este un identificator secvențial independent de data reală. Managerul poate închide mai multe zile în aceeași seară. Dashboard-ul afișează doar comparații agregate PER ZI (inclusiv top produse și jocuri), nu intervale orare. Nu persistăm currentSimulatedMinute, openedAtSimulatedMinute, closedAtSimulatedMinute sau daily_occupancy_stats.

16.3. Raport PDF

Managerul poate exporta raportul FINALIZED în PDF. Recomandarea actuală este generarea PDF-ului la cerere din snapshot-ul persistent, fără tabel de fișiere PDF pentru raport. Include venituri, sesiuni, produse, jocuri, eticheta zilei și data generării.

17. Clean Code, SOLID și testare

17.1. Principii obligatorii de calitate

Principiu	Aplicare proiect
S — Single Responsibility	Servicii pe capabilități: rezervări, checkout, plată, raportare; controller subțire.
O — Open/Closed	Schimbarea furnizorului AI sau a strategiei de recomandare prin contracte adecvate, fără modificarea Core.
L — Liskov Substitution	Implementările contractelor respectă aceleași precondiții și rezultate observabile.
I — Interface Segregation	Interfețe mici numai unde ajută; fără „interfață pentru fiecare clasă”.
D — Dependency Inversion	Dependențe de abstracții potrivite pentru AI, mesagerie sau alte integrări externe.
Clean Code	Nume precise, metode scurte, un nivel clar de abstracție, validări explicite, excepții gestionate, fără duplicate.
JPA & API	DTO-uri separate de entități; fără expunere directă a modelului persistent.
Frontend	Componente UI separate de API access, state transitions și business validation pe server.

17.2. Strategia de testare

Nivel	Scenarii esențiale
Unit	Calcul nota, limite jocuri, validări status, filtrare deterministă, agregarea zilnică.
Integrare PostgreSQL	Unicitate condiționată, FK, constrângeri, migrații Flyway, tranzacții.
Concurență	Dublă rezervare, ultimele sloturi de livrare, comandă vs checkout, plată dublă, reopen vs payment, EOD vs open session.
API & securitate	RBAC, participare limitată la sesiune, PIN greșit/rate-limit, token vechi după CLOSED, acces neautorizat la preview.
End-to-end	Laptop staff + iPhone client; QR, meniu, cerere joc, plată și raport EOD.
Fallback AI	AI indisponibil: recomandări filtrate, fără explicații AI, Core funcțional.
Reconnect	Staff/customer pierd WebSocket și recuperează starea prin REST.

17.3. Criterii de review înainte de merge

- Clasa are o responsabilitate clară, nume comprehensibile și dependențe justificate.
- Regulile de business rămân în backend; validările UI nu constituie protecții de securitate.
- DTO-urile nu expun tokenuri, PIN-uri, hash-uri sau câmpuri interne.
- Orice operație compusă este testată la rollback și concurență.
- Migrațiile păstrează datele existente; API-ul are erori previzibile și loguri fără secrete.
- Se poate explica de ce există fiecare clasă, metodă, query și anotare JPA.

18. Inițializare și demonstrație

18.1. Bootstrap fără editări manuale

Sistemul trebuie să poată deveni utilizabil după instalare, fără INSERT manual în PostgreSQL. Mecanismul exact de bootstrap este încă DESCHIS; trebuie să creeze/permită setarea primului cont MANAGER, CafeSettings

și BusinessDay #1 OPEN. Datele demonstrative pentru mese, jocuri, copii, categorii și produse trebuie să fie create prin seed controlat sau interfețe administrative.

Artefact inițial	Regulă
Primul manager	Bootstrap sigur, credențiale inițiale schimbate; metoda concretă de stabilit.
CafeSettings	Un set inițial de tarife, limită jocuri și nume cafenea.
BusinessDay #1	OPEN la prima pornire; nu se recreează după restart.
Date demo	Seed idempotent în profil demo; nu dublează inventarul după restart.
PDF-uri reguli	Upload din UI; eventual fișiere demo licențiate/permise, neincluse implicit.

18.2. Scenariul demonstrativ principal

1. Pornire aplicații și autentificare ca staff; deschidere Masă 1 și acces la preview QR/PIN.
2. Scanare pe iPhone, introducere PIN și nume; al doilea participant intră la aceeași masă.
3. Plasare comandă cu produse acceptate și respinse; actualizări WebSocket pe client și staff.
4. Solicitare joc, rezervarea atomică a unui exemplar, livrare și returnare.
5. Declararea persoanelor fizice; checkout; blocarea tarifelor; redeschidere de către staff și corecție.
6. Plată simulată integrală, închiderea sesiunii; vechiul QR nu mai poate fi folosit.
7. Închiderea tuturor meselor, End of Day, generarea snapshot-ului și Ziua 2 OPEN.
8. Repetarea fluxului pentru a produce grafice comparative între zile; export raport PDF.
9. Demonstrarea AI recommendation + fallback și a întrebărilor pe regulile PDF.
10. Arătarea a două teste de concurență și a diagramelor cerute la prezentare.

19. Registrul deciziilor (ADR)

ID	Decizie / justificare scurtă	Statut
D-01	Clienți fără cont; staff/manager autentificați	CONFIRMAT
D-02	O sesiune neînchisă per masă; QR/PIN noi per sesiune	CONFIRMAT
D-03	Preview QR/PIN per masă în Management; tablete fizice la Future Work	CONFIRMAT
D-04	Coș separat și comenzi separate per participant, notă comună	CONFIRMAT
D-05	Acceptare/respingere per produs; toate respinse → CANCELLED, fără taxare	CONFIRMAT
D-06	GameCategory N:M prin game_category_links	CONFIRMAT
D-07	Rezervarea exemplarului la acceptarea cererii, fără dublă alocare	CONFIRMAT
D-08	Numai jocurile DELIVERED contează la limita mesei	CONFIRMAT
D-09	Fără expirarea automată a rezervărilor	CONFIRMAT
D-10	Tarif per persoană fixat în TableSession la deschidere	CONFIRMAT
D-11	Cantitatea fizică declarată manual; locked după primul checkout	CONFIRMAT
D-12	Plată simulată unică integrală; închidere automată	CONFIRMAT
D-13	BusinessDay secvențial, EOD manual exclusiv manager, zero sesiuni deschise	CONFIRMAT
D-14	Snapshot final per zi cu statistici pe produse și jocuri, PDF export	CONFIRMAT

ID	Decizie / justificare scurtă	Statut
D-15	Fără ceas virtual / fără statistici orare / fără daily_occupancy_stats	CONFIRMAT / ÎNLOCUIEȘTE ADR-14 VIRTUAL CLOCK
D-16	AI pe inventar eligibil + fallback non-AI; asistent reguli din PDF	CONFIRMAT
D-17	Backend Java Spring Boot; REST + WebSocket; React pentru UI	DIRECȚIE ACCEPTATĂ
D-18	PostgreSQL + JPA/Hibernate Code First; Flyway pentru migrații speciale	PROPUS / RECOMANDAT
D-19	Core + AI Service separat, fără mutații DB cross-service	PROPUS / RECOMANDAT
D-20	Docker, HTTPS și deploy pentru bonus	OBIECTIV; SOLUȚIE DESCHISĂ
D-21	Clean Code și SOLID obligatorii, fără overengineering	CONFIRMAT
D-22	PWA ca client mobil	DESCHIS: acceptarea profesorului

19.1. Corespondența ADR anterioare

ADR-11 anterior = strategia Code First; ADR-12 = QR/PIN și preview; ADR-13 = Clean Code/SOLID; ADR-14 (ceas virtual) a fost RESPINS explicit când s-a decis raportarea doar pe zile. Prezenta consolidare folosește identificatori D-xx stabili, pentru a evita reutilizarea ambiguă a numerelor istorice.

20. Riscuri, puncte deschise și Future Work

20.1. Decizii deschise înainte de înghețarea DB v1.0

Prioritate	Element	Ce trebuie decis
P0	Validarea cerinței mobile	Profesorul acceptă React PWA drept aplicație mobilă cu 3 operații sau este necesar alt client?
P0	Mecanismul exact QR/PIN	Tip token, hash/digest + criptare, rotația cheilor, rate limit, protecții cookie, intrare în checkout.
P0	Inițializarea primului manager	Bootstrap sigur, prima zi, setări globale și seed idempotent.
P0	Topologia deploy și buget	Hosting backend-uri, frontend, PostgreSQL, PDF-uri și HTTPS; cost de exploatare.
P1	AI model și hosting	API extern vs model local, limite, cost, latență și fallback.
P1	Coșul de cumpărături	React local propus; păstrarea cross-device nu este confirmată.
P1	Modul de procesare a pozițiilor	Confirmarea atomică a întregii decizii per comandă și livrare comună sunt propuse.
P1	Cererile rezervate la checkout	Staff-ul poate finaliza/anula rezervările existente; politica exactă de livrare în checkout de închis.
P1	Corecția tarifelor	Confirmat după redeschidere; modificarea directă în CHECKOUT_PENDING nu este aprobată.
P1	Număr categorii joc	Minimum o categorie activă pentru joc publicat este propus, nu confirmat.
P1	Reguli PDF	Staging + activare atomică a noului document; stocarea și versionarea tehnică exacte de ales.
P2	Statistici suplimentare	Peak concurrent tables/bon mediu/categorii per zi nu sunt obligatorii în schema actuală.
P2	Orarul cafenelei	Program informativ, fără automatizări de deschidere; cazuri peste miezul nopții în afara scope-ului.

20.2. Future Work explicit

- Tablete fizice, autentificarea dispozitivelor şi asocierea device–masă printr-un posibil table_devices.
- Plăţi reale, reduceri, plăţi parţiale, bon fiscal/facturi şi rambursări.
- OCR pentru PDF-uri scanate, embeddings/vector search dacă soluţia simplă nu este suficientă.
- Grafic de intervale aglomerate / ceas virtual doar dacă cerinţele viitoare se schimbă (EXCLUS acum).
- Conturi persistente de client, istoric între vizite, rezervări online în afara sesiunilor.

20.3. Riscuri și măsuri

Risc	Impact	Răspuns planificat
PWA neacceptată ca mobile	Neacoperirea cerinţei academice	Clarificare timpurie cu profesorul; plan alternativ client mobil.
Scope prea mare pentru un singur dezvoltator	Întârziere / produs incomplet	Vertical slices; funcţiile minim demonstrabile înaintea bonusurilor.
AI extern lent/scump	Costuri / indisponibilitate	Timeout, fallback, plan de testare şi buget.
Dublări în rezervări/plăţi	Date invalide	Constrângeri DB + tranzacţii + teste concurente.
QR neaccesibil pe telefon	Demo blocat	HTTPS public sau reţea locală accesibilă, test pe iPhone.
Flyway / schema drift	Eroare la deploy	Migraţii versionate şi validate în CI.
Raport EOD inconsistent	Date demonstrative greşite	Unicitate snapshot, calcul server-side şi operaţie atomică.

21. Plan de lucru și criterii de acceptare

21.1. Secvența recomandată a implementării

Etapă	Rezultat verificabil
Faza 0	Înghețare DB v1.0, clarificare PWA, bootstrap, contracte API, diagrame de proiect.
Faza 1	Skeleton Spring + PostgreSQL + auth staff; prima zi/manager/setări; CRUD mese şi preview QR/PIN.
Faza 2	Customer join pe iPhone + SessionParticipant + refresh/reconnect, WebSocket bază.
Faza 3	Catalog meniu, coş, comenzi şi acceptare parţială; teste.
Faza 4	Catalog jocuri, categorii N:M, copies, cereri şi tranzacţii de rezervare/livrare/returnare.
Faza 5	Tarife, checkout, redeschidere şi plată simulată; teste de concurenţă.
Faza 6	BusinessDay, End of Day, analitice comparate şi export PDF.
Faza 7	AI Service: recomandări/reranking + fallback; PDF rules retrieval.
Faza 8	Docker/deploy/HTTPS, client mobil demonstrat, teste E2E, diagrame şi prezentare.

21.2. Definition of Done — nivel proiect

- Se pot crea date şi parcurge toate fluxurile fără alterarea manuală a DB.
- Minimum 8 tabele relaţionate valide academic; minimum 12 CRUD endpoint-uri şi 5 complexe, două custom queries.
- Minimum 10 funcţionalităţi frontend, trei complexe, server activ prin WebSocket, client mobil acceptat de profesor cu minimum 3 operaţii.
- Integrare AI reală + fallback, asistent PDF, eroare AI izolată de Core.
- QR scanabil pe iPhone, sesiune distinctă per masă, preview reafişabil, revocarea QR după CLOSED.

- Inventar fără dublă rezervare, limita DELIVERED respectată, o singură plată integrală, EOD fără dublări.
- Rapoarte PER ZI, snapshot-uri complete şi export PDF; nicio statistică orară în v1.
- Testele critice şi diagramele (bloc, use-case, secvenţă) sunt pregătite pentru prezentare.
- Cod explicabil, separare responsabilităţi, DTO-uri, SOLID şi migraţii controlate; deploy dacă este fezabil.

Anexe — Glosar

Termen	Sens în proiect
BusinessDay	Zi simulată numerotată; asociere logică, nu zi calendaristică.
TableSession	Utilizarea unei mese de un grup, cu propriile coduri de intrare şi notă comună.
SessionParticipant	Identitate temporară de client în sesiunea unei mese.
GameCopy	Exemplar fizic identificabil al unui titlu din catalog.
GameRequest	Cererea unui titlu, cu posibilă alocare de copie şi istoric de împrumut.
Snapshot	Valori istorice păstrate în forma definită la eveniment; nu recalcularea din preţuri curente.
Code First	Entităţile Java sunt punctul de plecare al modelului; schema este controlată în timp şi prin migraţii.
Flyway	Instrument de migrare versionată a bazei de date.
JPA/Hibernate	Mapare între obiectele Java şi tabelele relaţionale.
Idempotency	Repetarea aceleiaşi cereri nu produce dublarea efectului.
Optimistic locking	Detectarea modificărilor concurente printr-un număr de versiune.
REST	API HTTP pentru comenzi şi recuperare a stării curente.
WebSocket	Canal pentru notificări server către frontend.
RAG	Răspuns asistat de fragmente recuperate din surse documentare, aici reguli PDF.
DTO	Obiect pentru cereri/răspunsuri API, separat de entitatea JPA.
Future Work	Capabilitate documentată, neimplementată în v1.

Anexe — Reconcilierea deciziilor înlocuite

Propunere mai veche	Decizia finală curentă
Tablete fizice în fiecare masă + table_devices	Preview QR/PIN per masă în Management; tablete reale la Future Work.
O singură categorie per joc	Categorii N:M via game_category_links.
Tarif preluat la prima introducere / checkout	Tarif per persoană fixat la deschiderea sesiunii.
Raportare în funcţie de ore reale / intervale de business	Zile simulate secvenţiale, încheiate manual prin EOD.
Ceas virtual, currentSimulatedMinute, daily_occupancy_stats	Respins explicit; doar statistici zilnice.
Fee unit price şi în session_fees	Preţ snapshot în TableSession; quantity în SessionFee.
Stare permanentă isOccupied pe masă	Ocupare derivată din TableSession neînchisă.
availableCopies permanent în games	Disponibilitate derivată din GameCopy şi GameRequest.
Bill separat în schema DB	Notă calculată din sesiune, comenzi şi tarife; Payment păstrează suma finală.

Capăt document • Versiune Master v1.0 • 11 octombrie 2026